

Designing and Analysis of Algorithms

Course Code: ECS 5101/CS514

- Dr Rahul Mishra
- IIT Patna

- **Lecture 22**

Directed Graphs :-

④

A directed Graph G , also called **digraph** or **graph**, is the same as a multigraph except that each edge e in G is assigned a direction, or in other words, each edge e is identified with an ordered pair (u, v) of nodes in G rather than unordered pair $[u, v]$.

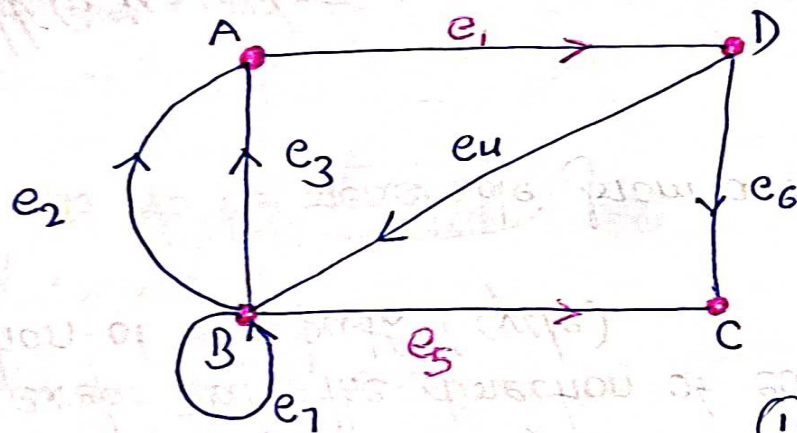
Suppose G is a directed graph with a directed edge $e = (u, v)$. Then e is also called an arc.

- (1) e begins at u and ends at v .
- (2) u is the origin or initial point of e , and v is the destination or terminal point of e .
- (3) u is a predecessor of v , and v is a successor or neighbor of u .
- (4) u is adjacent to v , and v is adjacent to u .

- * The outdegree of a node u in G , written $\text{outdeg}(u)$, is the number of edges beginning at u .
- * Similarly, the indegree of u , written $\text{indeg}(u)$, is the number of edges ending at u .
- * A node u is called a source if it has a positive outdegree but zero indegree.
- * A node u is called a sink if it has a zero outdegree but a positive indegree.
- * The notions of path, simple path, and cycle carry over from undirected graphs to directed graphs except that the direction of each edge in a path (cycle) must agree with the direction of the path (cycle).
- * A node v is said to be reachable from a node u if there is a (directed) path from u to v .
- * A directed graph G is said to be connected or strongly connected, if for each pair u, v of nodes in G there is a path from u to v and there is also a path from v to u .

* G is said to be *unilaterally connected* if any pair u, v of nodes in G there is a path from $\boxed{u \text{ to } v}$ or a path from $\boxed{v \text{ to } u}$.

⑥



A directed graph G with 4 nodes and 7 (directed) edges.

- (iv) $P_1 = (D, B, A)$ is a path
- (v) G is unilaterally connected.
- (vi) $\text{indeg}(D) = 1$
- (vii) $\text{outdeg}(D) = 2$
- (viii) C is a sink
- (ix) No node in G is source.

- (i) edges e_2 and e_3 are parallel (begin B end A)
- (ii) e_7 is a loop: starts and ends at same point
- (iii) The sequence $P_1 = (D, C, B, A)$ is not a path, since (C, B) is not an edge. i.e; the direction of edge $e_5 = (B, C)$ does not agree with direction

Adjacency Matrix ÷

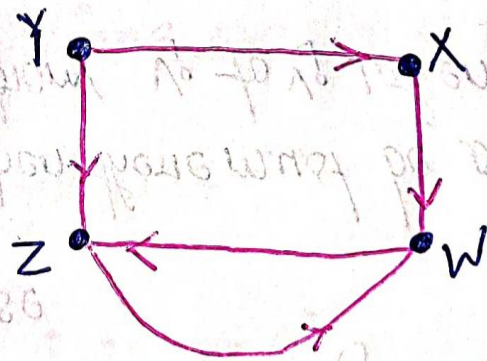
- Suppose G is a simple directed graph with m nodes, and suppose the nodes of G have been ordered and are called $v_1, v_2, \dots, v_m$. The adjacency matrix $A = (a_{ij})$ of the graph G is the $m \times m$ matrix defined as follows:

$$a_{ij} = \begin{cases} 1 & \text{if } v_i \text{ is adjacent } v_j, \text{ i.e., if there is an edge } (v_i, v_j) \\ 0 & \text{otherwise} \end{cases}$$

- Such a matrix A , which contains entries of only 0 and 1, is called a bit matrix or a Boolean matrix.
- The adjacency matrix A of the graph G does depend on the ordering of the nodes of G , i.e., a different ordering of the nodes may result in a different adjacency matrix.
- The matrices resulting from two different orderings are closely related in that one can be obtained from the other by simply interchanging rows and columns.

- * Suppose G is an undirected graph. The adjacency matrix A of G will be a symmetric matrix, i.e., one in which $\boxed{a_{ij} = a_{ji}}$ for every i and j .
- * This follows from the fact that each undirected edge $[u, v]$ corresponds to the two directed edges (u, v) and (v, u) .
- * Such matrix representation of a graph may be extended to multigraphs. Specifically, if G is a multigraph, then the adjacency matrix of G is the $n \times n$ matrix $A = (a_{ij})$ defined by setting a_{ij} equal to the number of edges from $\boxed{v_i \text{ to } v_j}$.

$$A = \begin{matrix} & \begin{matrix} X & Y & Z & W \end{matrix} \\ \begin{matrix} X \\ Y \\ Z \\ W \end{matrix} & \begin{bmatrix} 0 & 0 & 0 & 1 \\ 1 & 0 & 1 & 1 \\ 1 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix} \end{matrix}$$



Proposition:-

(9A)

Let A be the adjacency matrix of a graph G . Then $a_k(i, j)$, the ' ij ' entry in the matrix A^k , gives the number of paths of length k from v_i to v_j .

Consider,

$$A = \begin{matrix} & \begin{matrix} X & Y & Z & W \end{matrix} \\ \begin{matrix} X \\ Y \\ Z \\ W \end{matrix} & \begin{bmatrix} 0 & 0 & 0 & 1 \\ 1 & 0 & 1 & 1 \\ 1 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix} \end{matrix} \quad A^2 = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 1 & 0 & 1 & 2 \\ 0 & 0 & 1 & 1 \\ 1 & 0 & 0 & 1 \end{bmatrix} \quad A^3 = \begin{bmatrix} 1 & 0 & 0 & 1 \\ 1 & 0 & 2 & 2 \\ 1 & 0 & 1 & 1 \\ 0 & 0 & 1 & 1 \end{bmatrix} \quad A^4 = \begin{bmatrix} 0 & 0 & 1 & 1 \\ 2 & 0 & 2 & 3 \\ 1 & 0 & 1 & 2 \\ 1 & 0 & 1 & 1 \end{bmatrix}$$

• Accordingly, in particular, there is a path of length 2 from v_4 to v_1 , there are two paths of length 3 from v_2 to v_3 , and there are three paths of length 4 from v_2 to v_4 .

* Suppose we now define the matrix B_n as follows:-

$$B_n = A + A^2 + A^3 + \dots + A^n$$

* Then the ij entry of matrix B_n gives the number of paths of length $\leq n$ or less from node v_i to v_j .

SHORTEST PATHS:-

(11)

PATH Matrix :-

Let G be a simple directed graph with n nodes, $v_1, v_2, \dots, v_n$. The path matrix or reachability matrix of G is the n -square matrix $P = (p_{ij})$ defined as follows:

$$p_{ij} = \begin{cases} 1 & \text{if there is a path from } v_i \text{ to } v_j \\ 0 & \text{otherwise} \end{cases}$$

- $\Rightarrow$ Suppose there is a path from v_i to v_j . Then there must be a simple path from v_i to v_j when $v_i \neq v_j$ or there must be a cycle from v_i to v_j when $v_i = v_j$.
- $\Rightarrow$ Since G has only n nodes, such a simple path must have length $n-1$ or less or such a cycle must have length n or less.

NOTE:- The transitive closure of a graph G is defined to be the graph G' such that G' has the same nodes as G and there is an edge (v_i, v_j) in G' whenever there is a path from v_i to v_j in G .

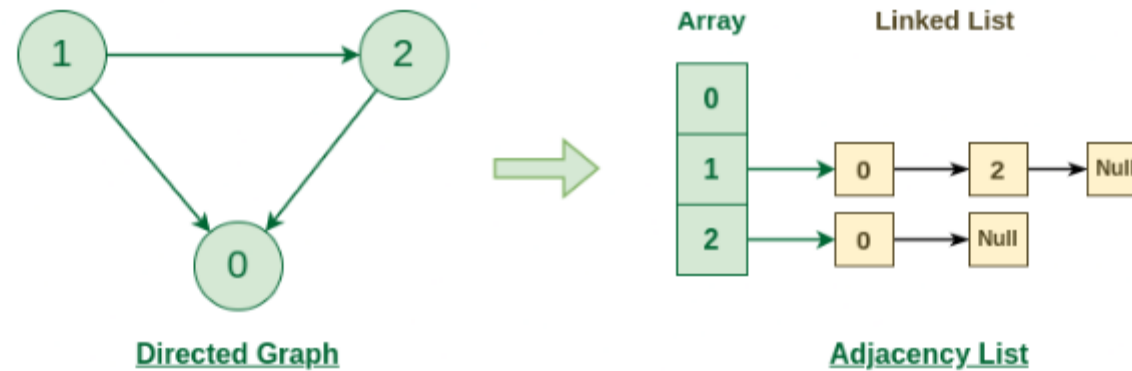
$\rightarrow$ Accordingly, the path matrix P of the graph G is precisely the adjacency matrix of its transitive closure G' .

LINKED REPRESENTATION OF A GRAPH

(19)

- Let G be a directed graph with n nodes. The sequential representation of G in memory — i.e., the representation of G by adjacency matrix A — has a number of major drawbacks.
- ① It may be difficult to insert and delete nodes in G . (as it requires change in size)
- ② If the number of edges is $O(n)$ or $(n \log_2 n)$ then the matrix A will be sparse; hence a great deal of space will be wasted.
- * G is usually represented in memory by a linked representation called as adjacency structure.

- An **adjacency list** is a data structure used to represent a graph where each node in the graph stores a list of its neighboring vertices.



Graph Representation of Directed graph to Adjacency List

How to build an Adjacency List?

It is very easy and simple to construct an adjacency list for a graph there are certain steps given below that you need to follow:

- Create an array of linked lists of size N , where N is the number of vertices in the graph.
- Create a linked list of adjacent vertices for each vertex in the graph.
- For each edge (u, v) in the graph, add v to the linked list of u , and add u to the linked list of v if the graph is undirected otherwise add v to the list of u if it is directed from u to v . (In case of weighted graphs store the weight along with the connections).

Applications of the Adjacency List:

- Graph algorithms: Many graph algorithms like Dijkstra's algorithm, Breadth First Search, and Depth First Search use adjacency lists to represent graphs.
- Image Processing: Adjacency lists can be used to represent the adjacency relationships between pixels in an image.
- Game Development: These lists can be used to store information about the connections between different areas or levels the game developers use graphs to represent game maps or levels.

Advantages of using an Adjacency list:

- An adjacency list is simple and easy to understand.
- Adding or removing edges from a graph is quick and easy.

Disadvantages of using an Adjacency list:

- In adjacency lists accessing the edges can take longer than the adjacency matrix.
- It requires more memory than the adjacency matrix for dense graphs.

OPERATIONS ON GRAPHS

- ① Searching in a Graph
- ② Inserting in a Graph
- ③ Deleting from a Graph

④ Traversing a Graph

DFS

BFS

Traversing a Graph:-

(28)

- » The breadth-first search will use a queue as an auxiliary structure to hold nodes for future processing, and analogously, the depth-first search will use a stack.
- » During the execution of our algorithm, each node N of G will be in one of three states, called the status of N , as follows:

STATUS = 1: (Ready state) The initial state of the node N .

STATUS = 2: (Waiting state) The node N is on the queue or stack, waiting to be processed.

STATUS = 3: (Processed state) The node N has been processed.

Breadth - First Search :-

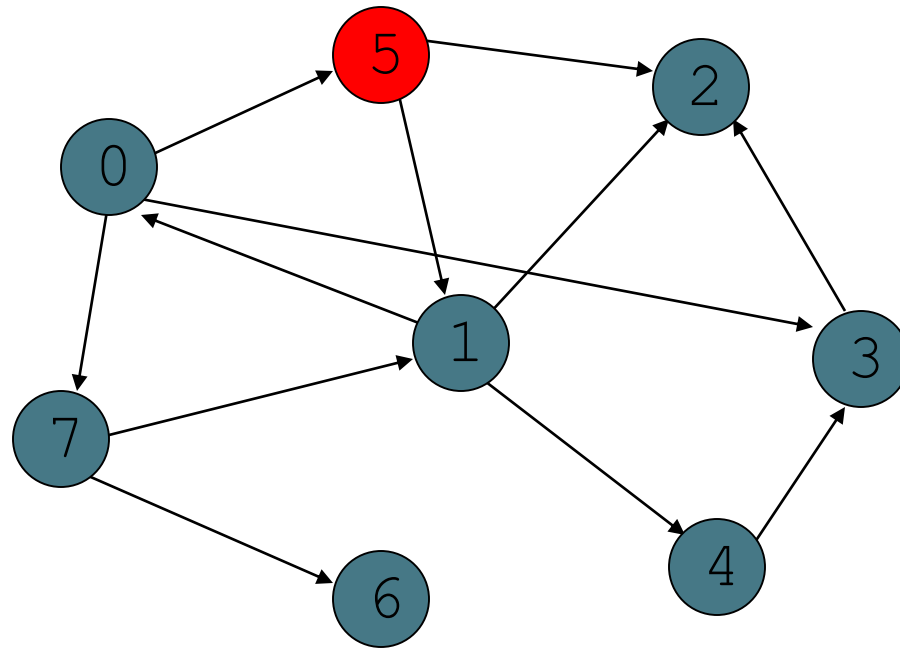
- * The general idea behind a breadth-first search beginning at a starting node A is as follows.
- * We examine the starting node A. (1)
- * Then, we examine all the neighbors of A. (2)
- * Then, we examine all the neighbors of the neighbors of A (3)
- * Thus, we need to keep track of the neighbors of a node, and we need to guarantee that no node is processed more than once.
- * It is accomplished by using a queue to hold nodes that are waiting to be processed, and by using a field STATUS which tells us the current status of any node.

Algorithm

This algorithm executes a breadth-first search on a graph G beginning at starting node A .

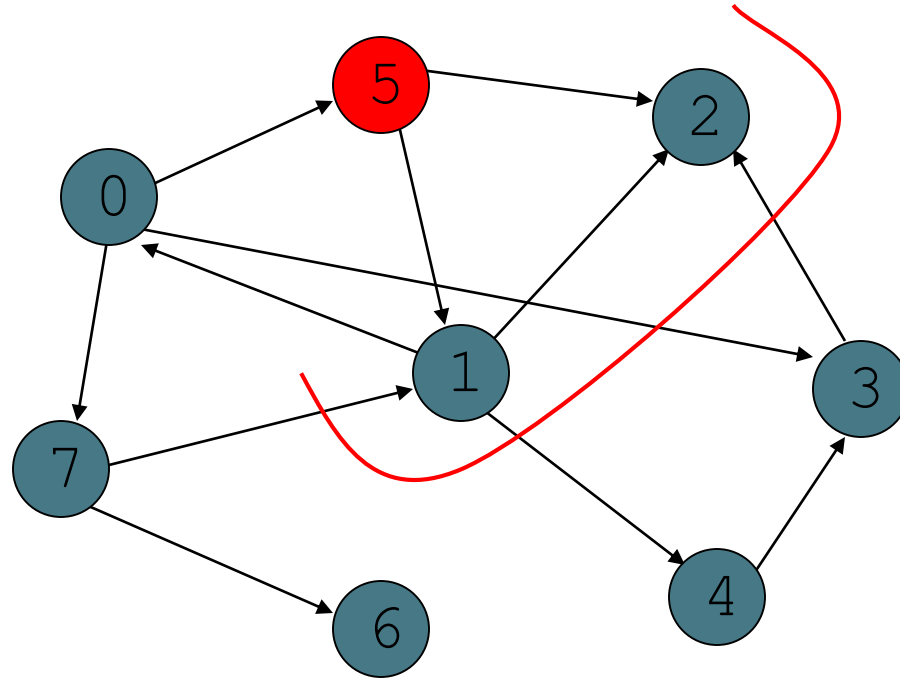
1. Initialize all nodes to the ready state ($STATUS = 1$).
2. Put the starting node A in $QUEUE$ and change its status to the waiting state ($STATUS = 2$).
3. Repeat steps 4 and 5
4. Remove the front node N of $QUEUE$. Process N and change the status of N to the processed state ($STATUS = 3$).
5. Add to the rear of $QUEUE$ all the neighbors of N that are in the steady state ($STATUS = 1$), and change their status to the waiting state ($STATUS = 2$).
6. Exit

BFS: Start with Node 5



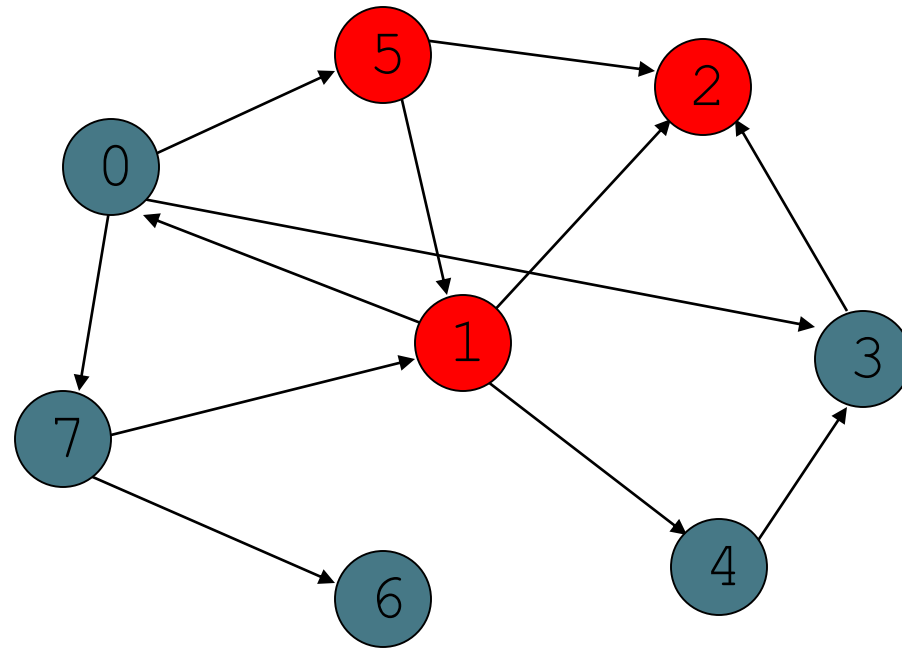
5

BFS: Node one-away



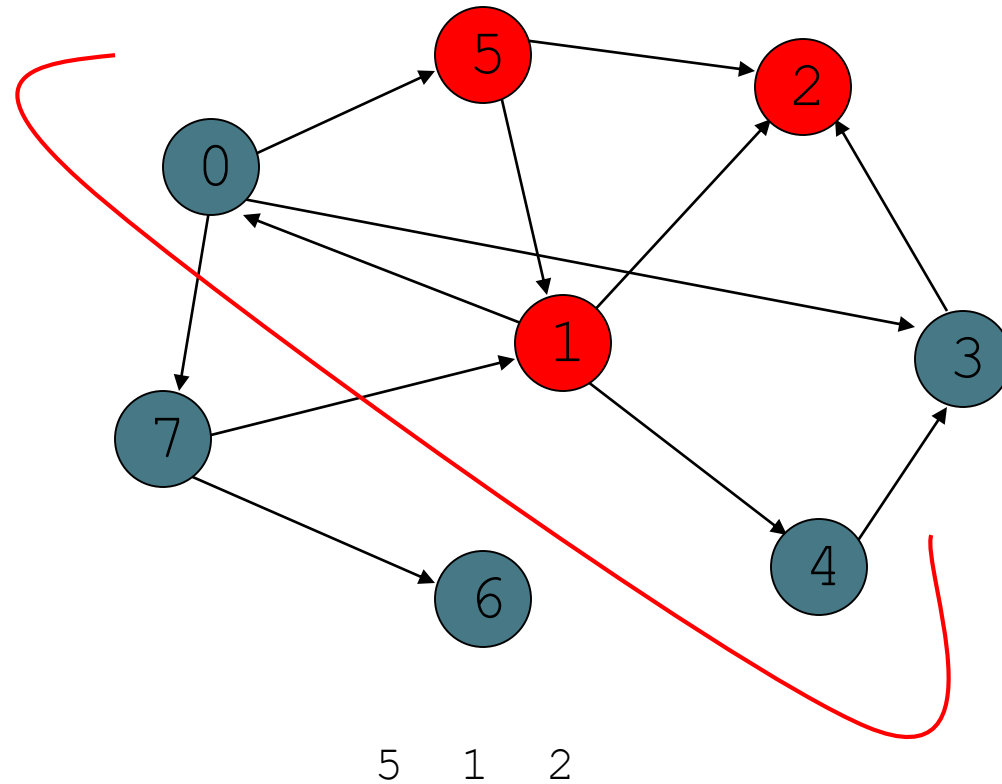
5

BFS: Visit 1 and 2

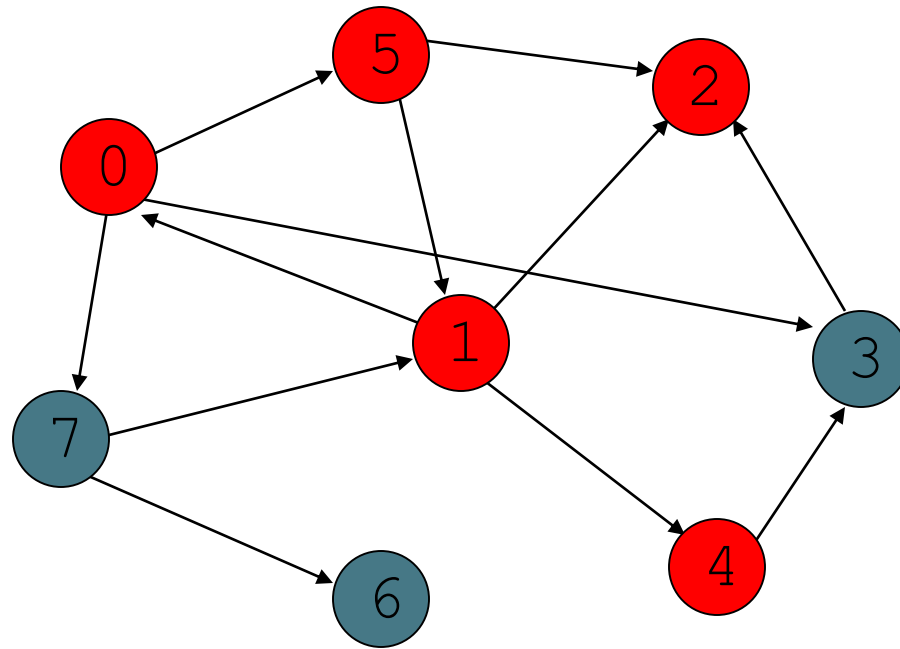


5 1 2

BFS: Nodes two-away

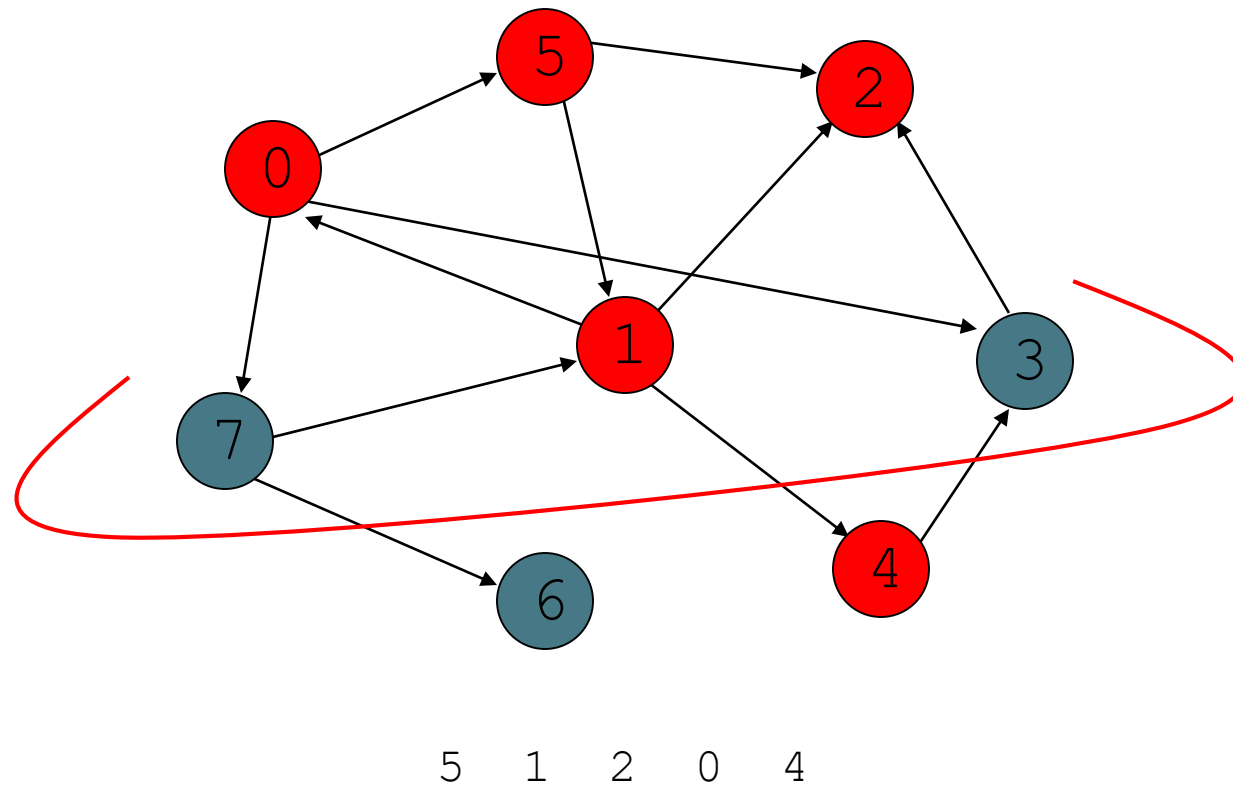


BFS: Visit 0 and 4

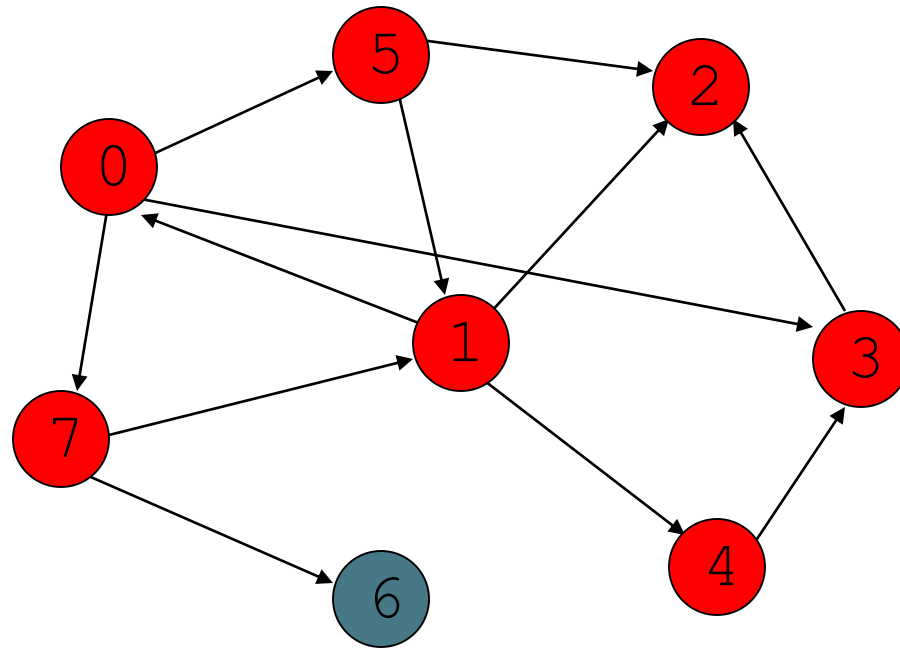


5 1 2 0 4

BFS: Nodes three-away

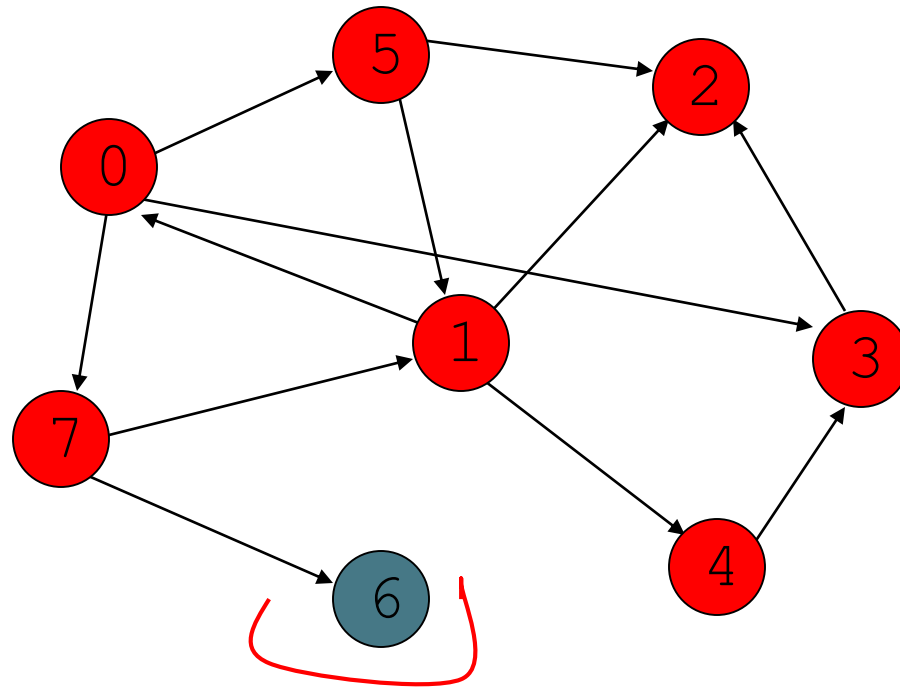


BFS: Visit nodes 3 and 7



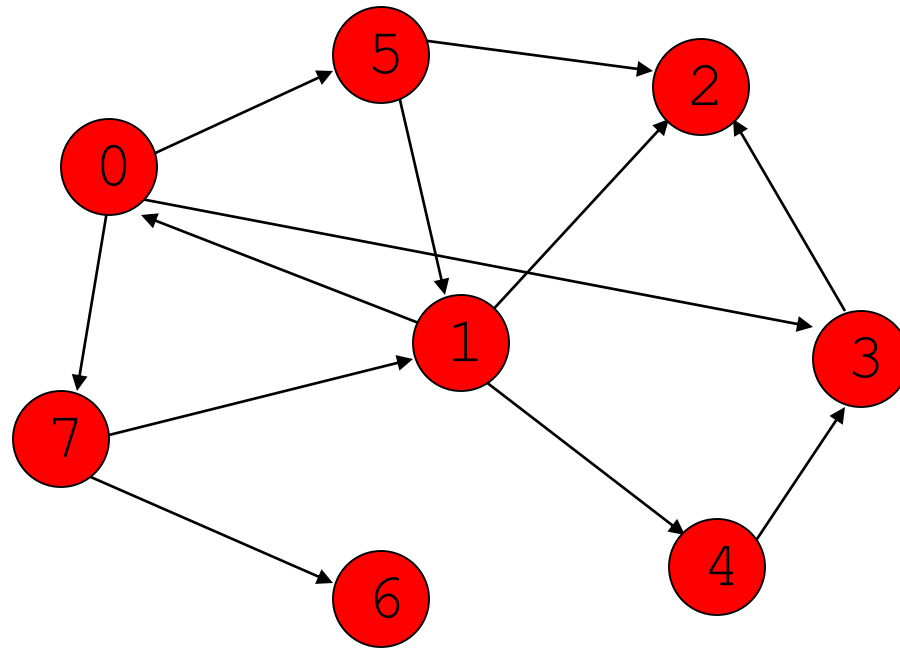
5	1	2	0	4
3	7			

BFS: Node four-away



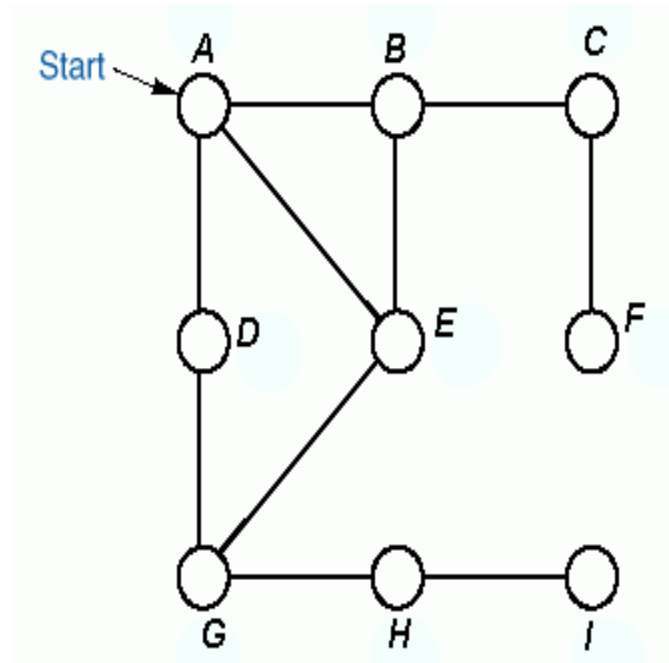
5 1 2 0 4
3 7

BFS: Visit 6

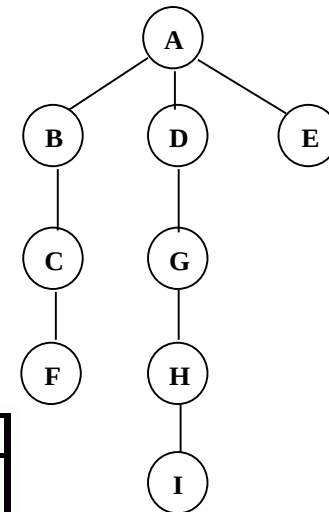


5	1	2	0	4
3	7	6		

Example of BFS



BFS-tree:



Order of
Traversal

1	2	3	4	5	6	7	8	9
A	B	D	E	C	G	F	H	I

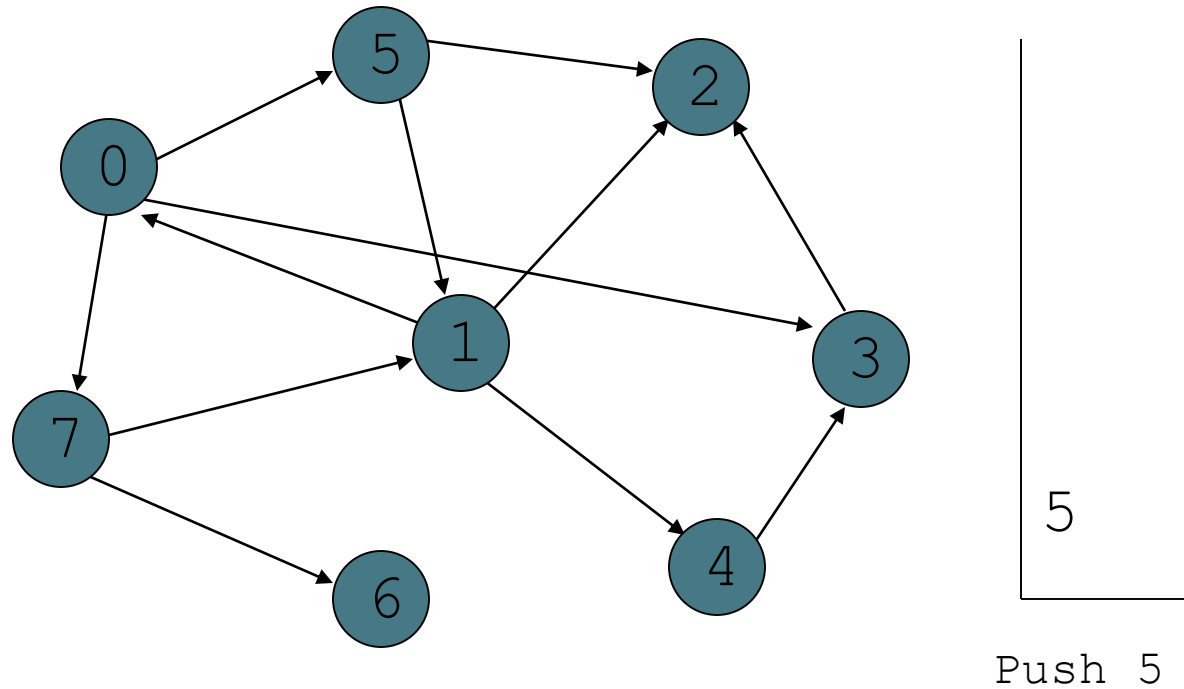
Depth-First Search :

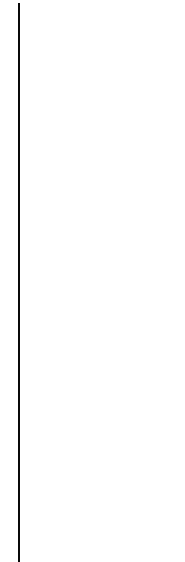
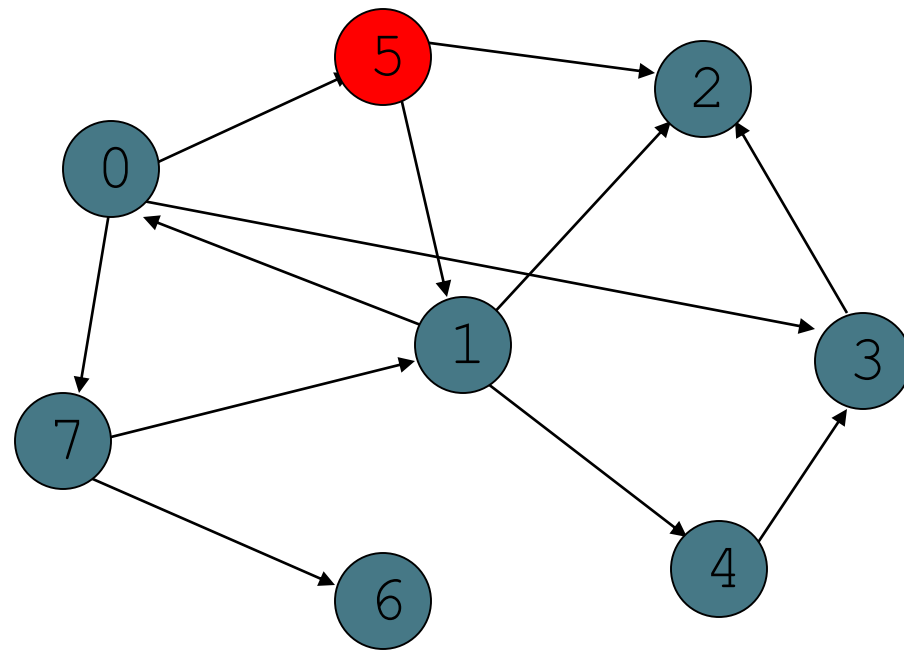
- ① A depth-first search beginning at a starting node A is as follows.
- ② First we examine the starting node A.
- ③ Then we examine each node N along a path P which begins at A; i.e., we process a neighbor of A, and so on.
- ④ After coming to a "dead end" that is, to the end of the path (P) we backtrack on (P) until we can continue along another, path P'. And so on
(“This algorithm is similar to the inorder traversal of a binary tree”)
- ⑤ This algorithm is very similar to the breadth-first search except now we use a stack instead of the queue.

Algorithm : This algorithm executes a depth-first search on a graph G beginning at a starting node A .

1. Initialize all nodes to the ready state ($STATUS=1$)
2. Push the starting node onto STACK and change its status to the waiting state ($STATUS=2$)
3. Repeat steps 4 and 5 until STACK is empty.
4. Pop the top node N of STACK. Process N and change its status to the processed state ($STATUS=3$)
5. Push onto STACK all the neighbors of N that are still in the ready state ($STATUS=1$), and change their status to the waiting state ($STATUS=2$)
6. Exit.

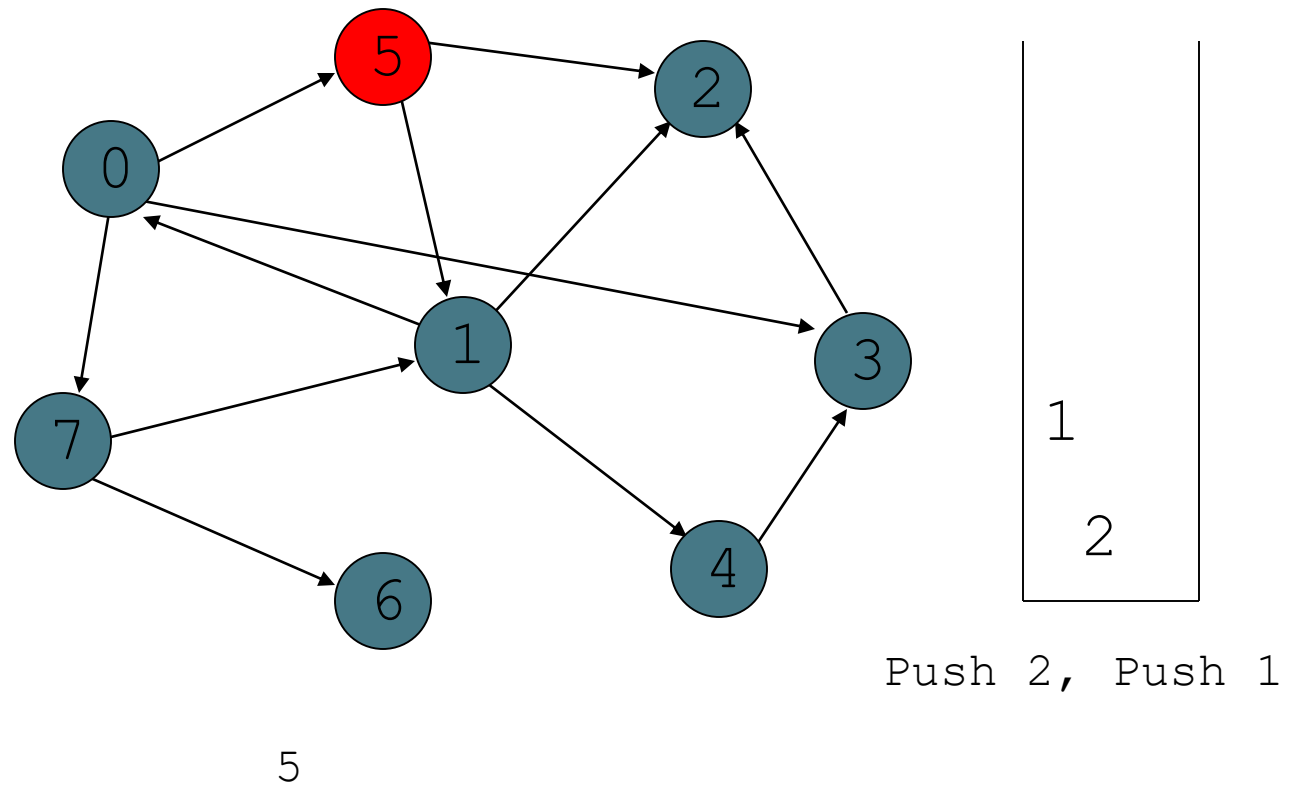
DFS: Start with Node 5

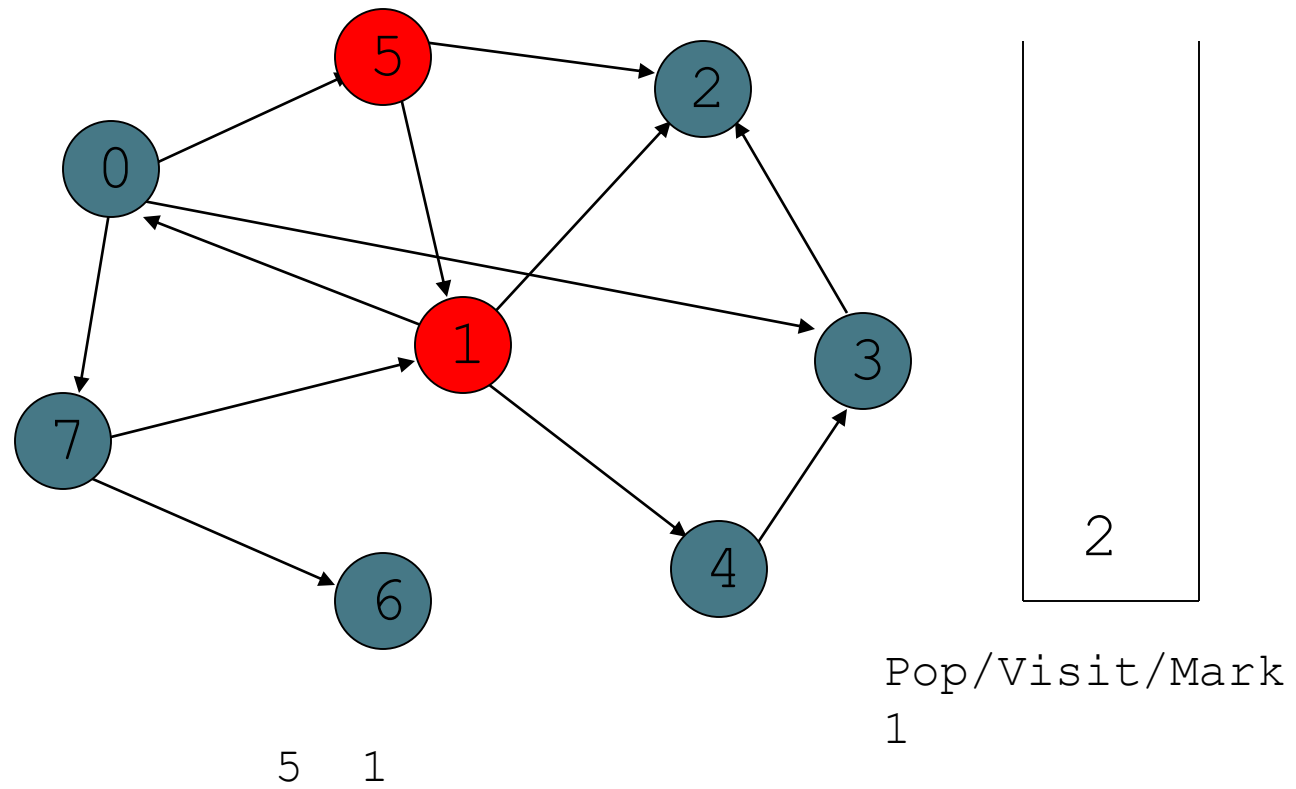


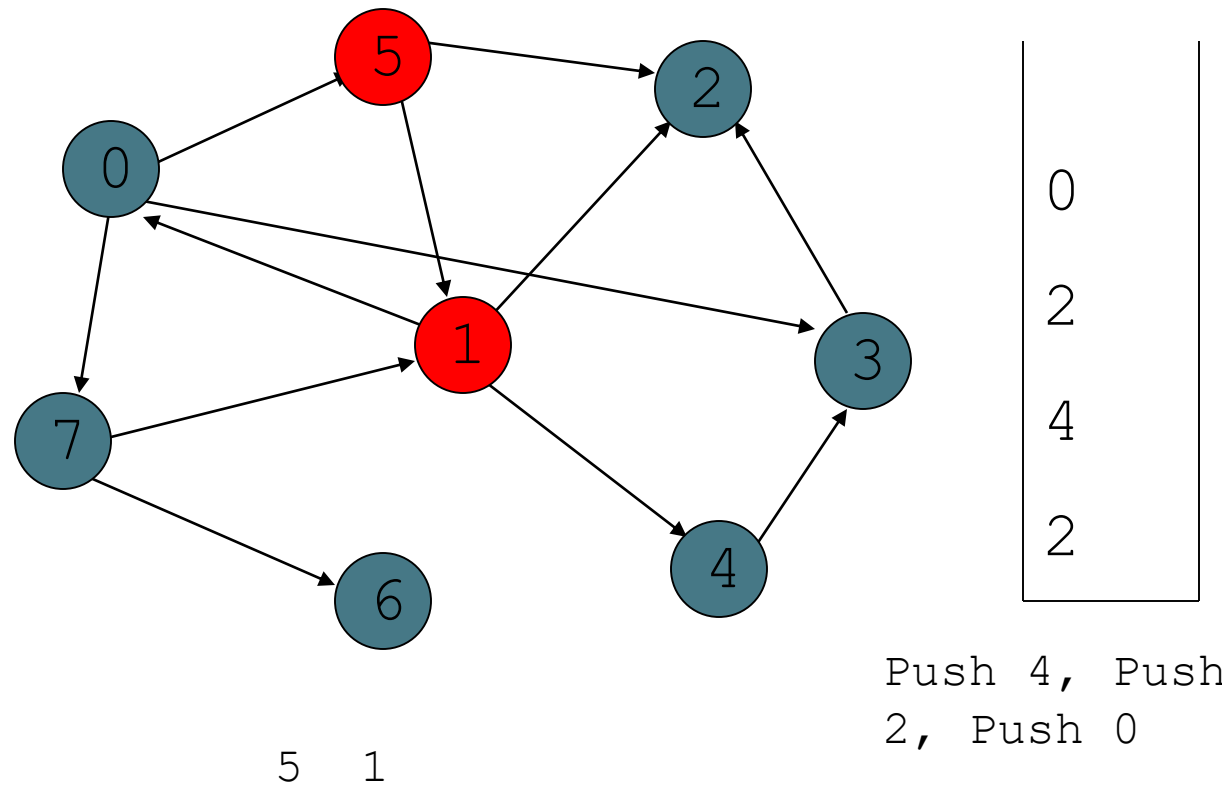


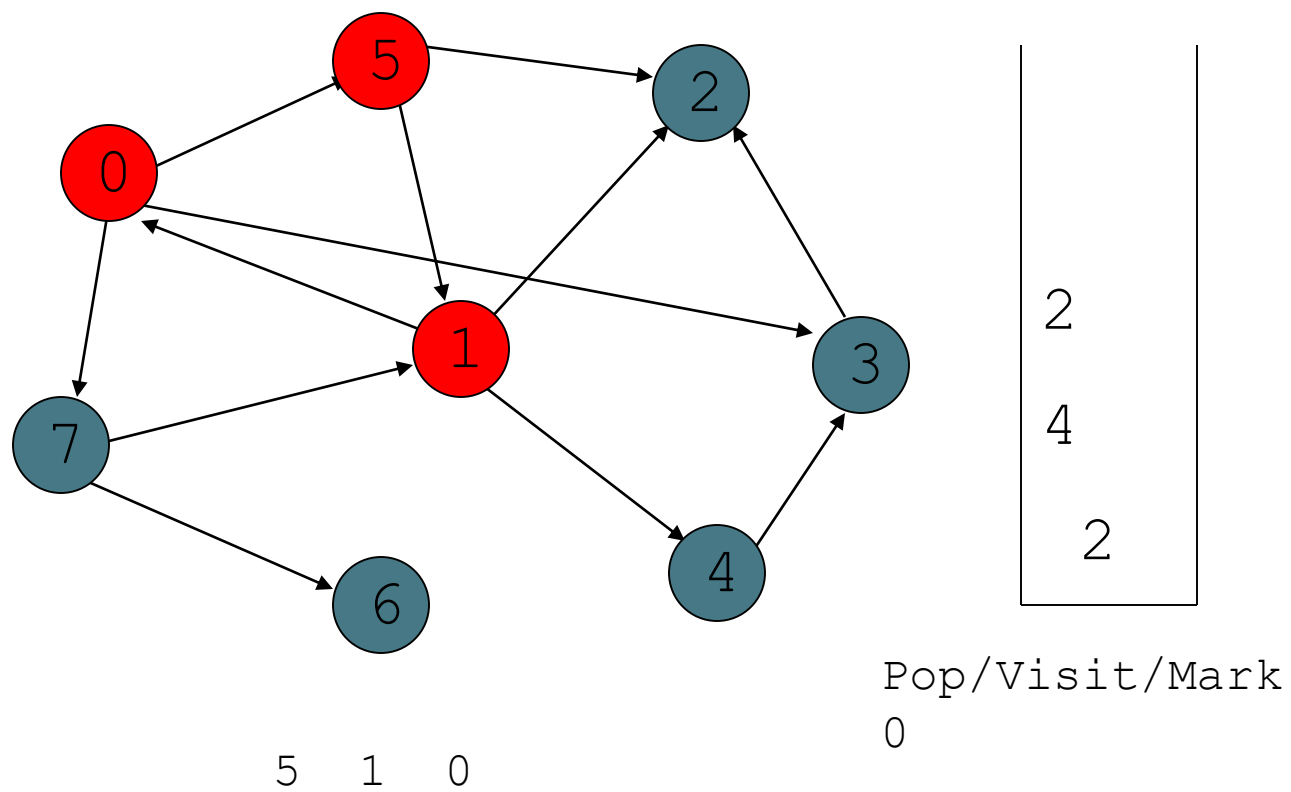
Pop/Visit/Mark
5

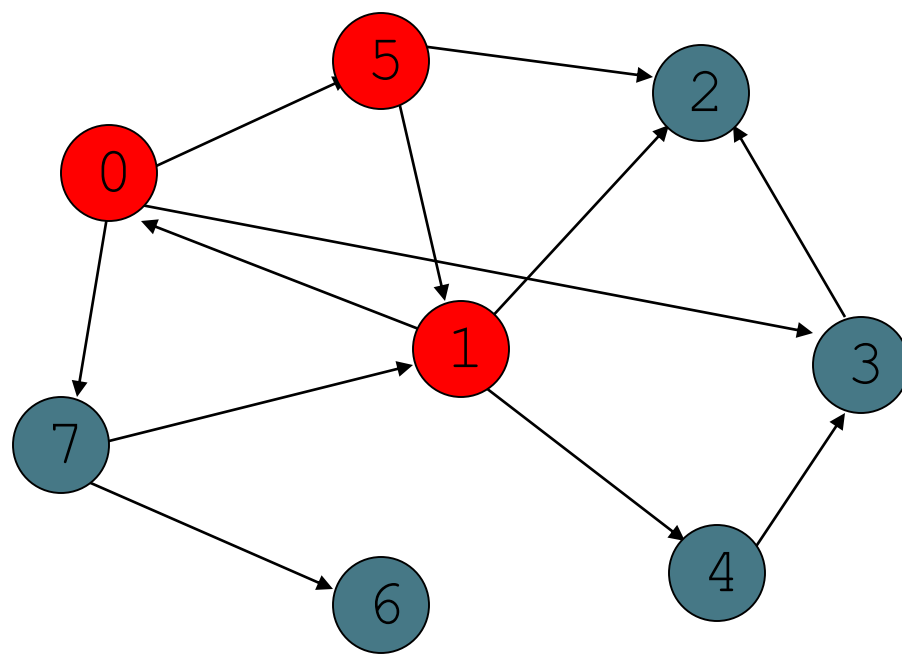
5







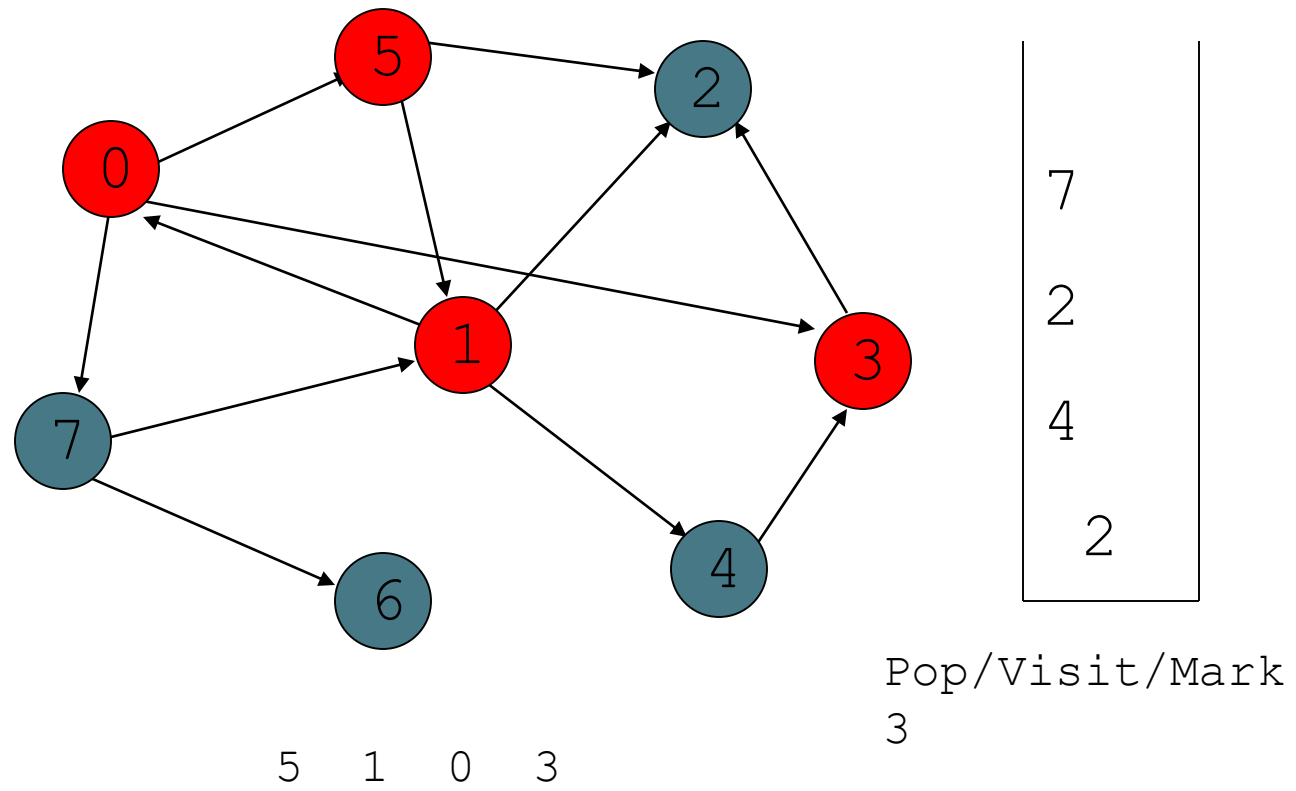


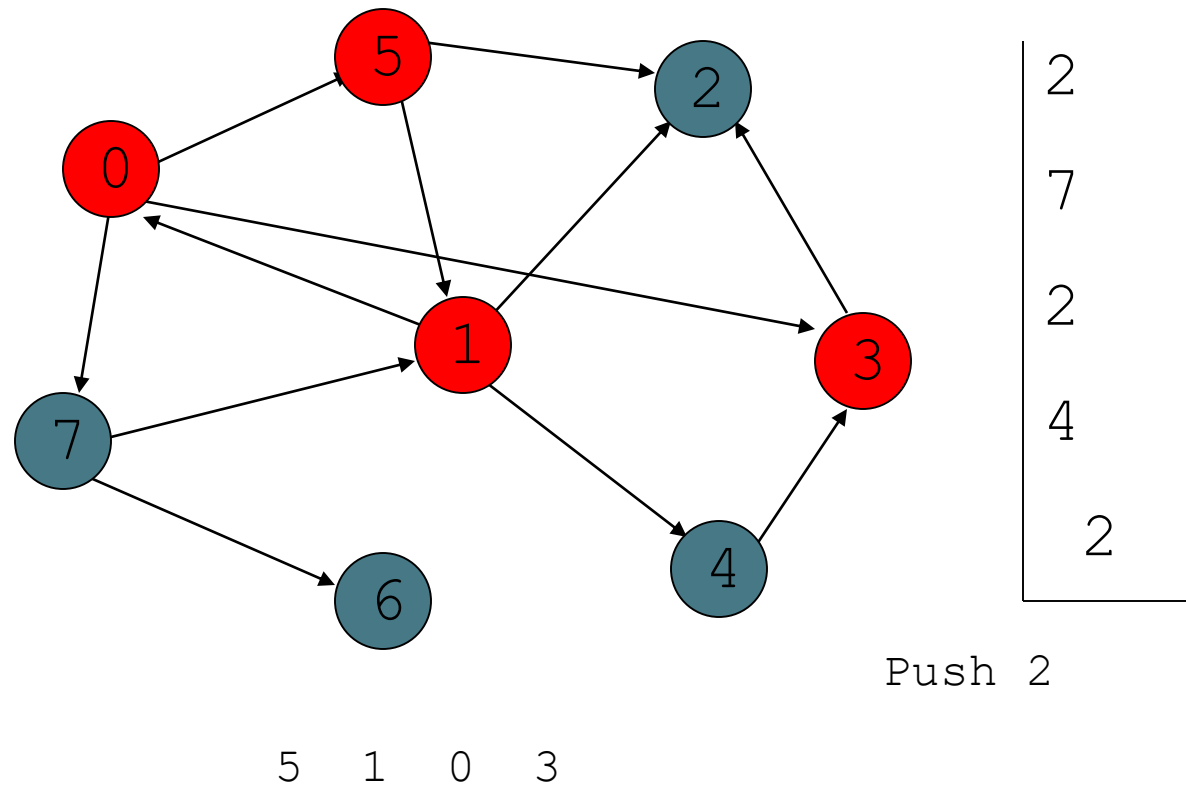


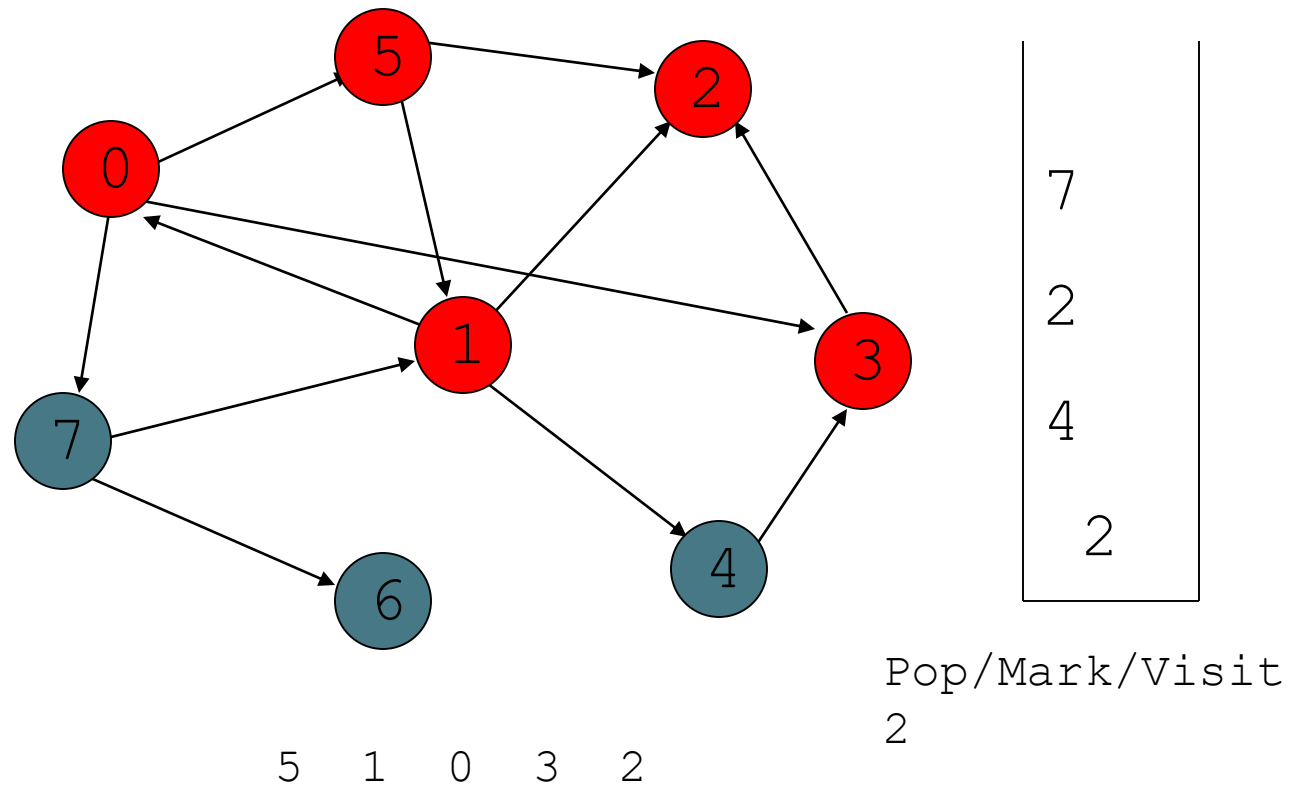
3
7
2
4
2

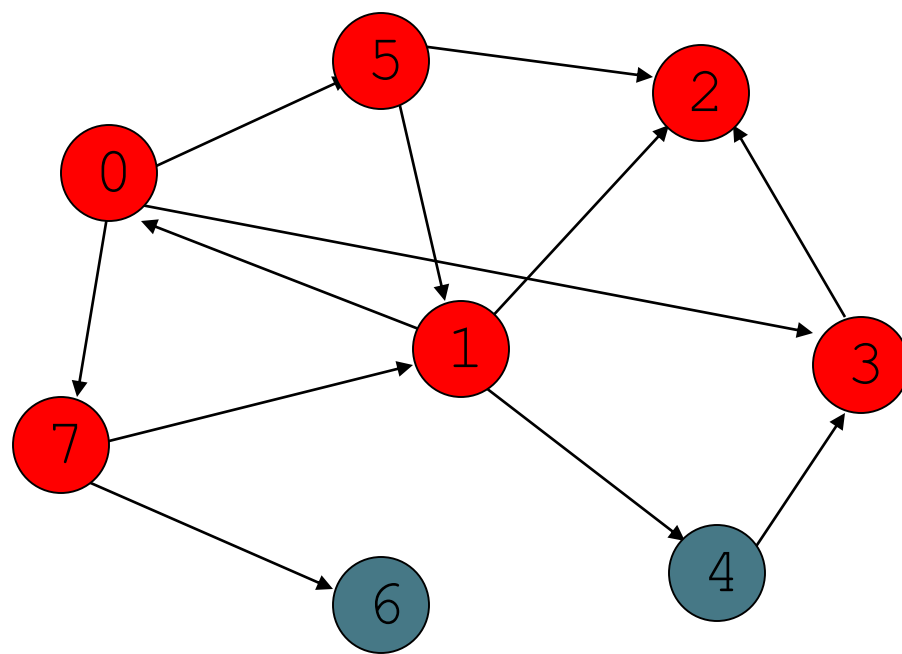
Push 7, Push 3

5 1 0





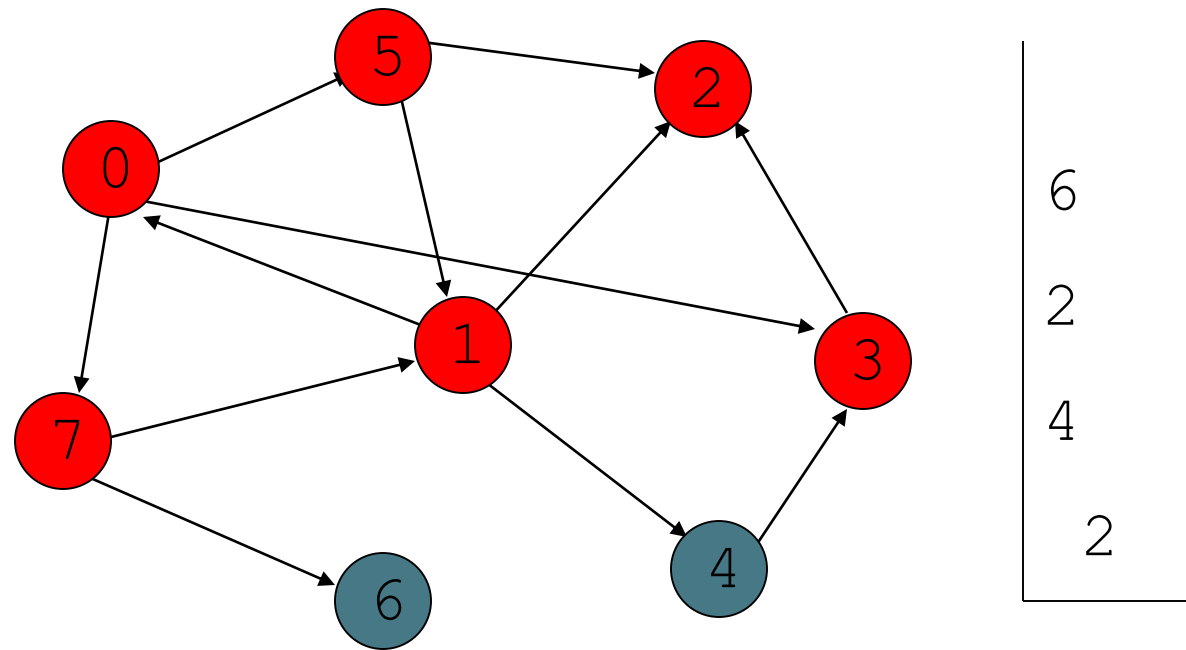




2
4
2

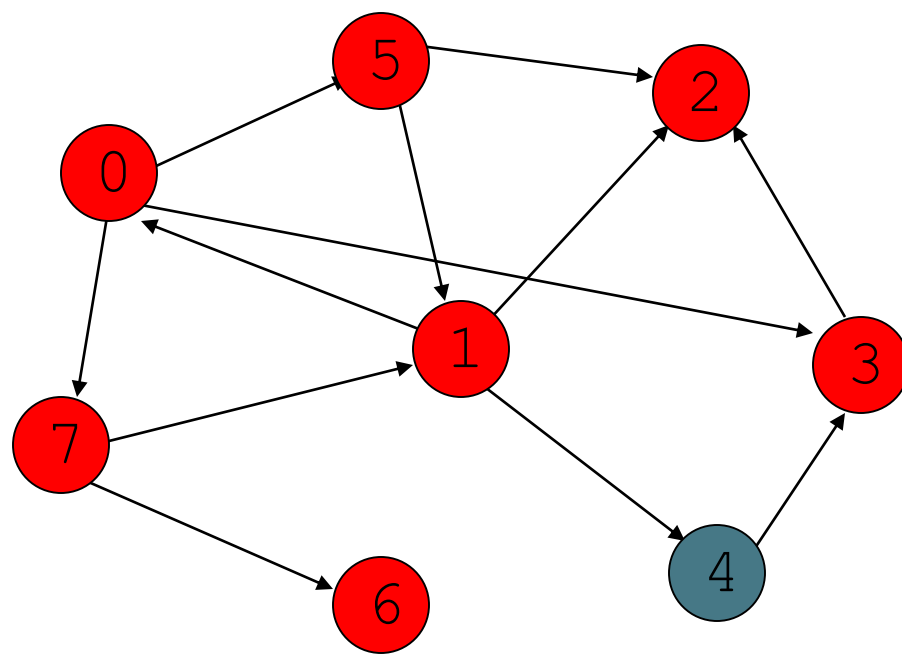
Pop/Mark/Visit
7

5 1 0 3 2
7



Push 6

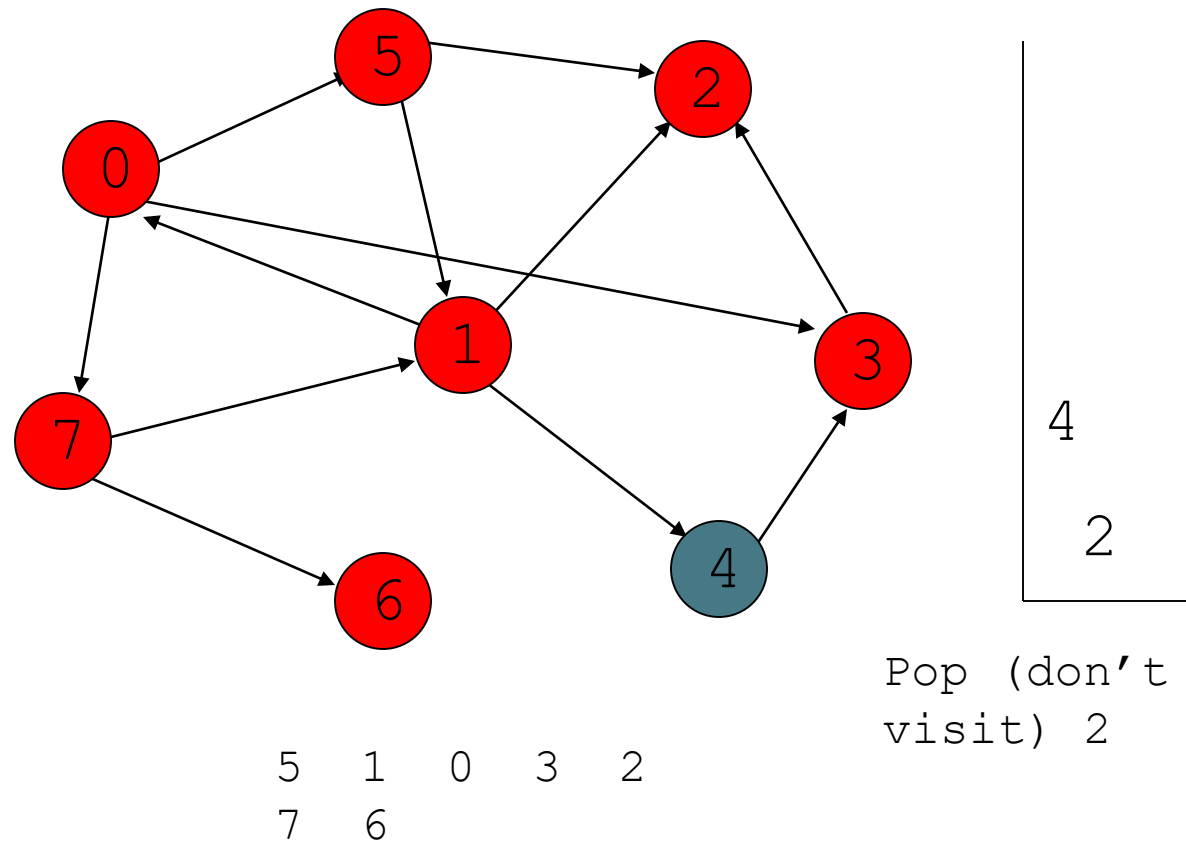
5 1 0 3 2
7



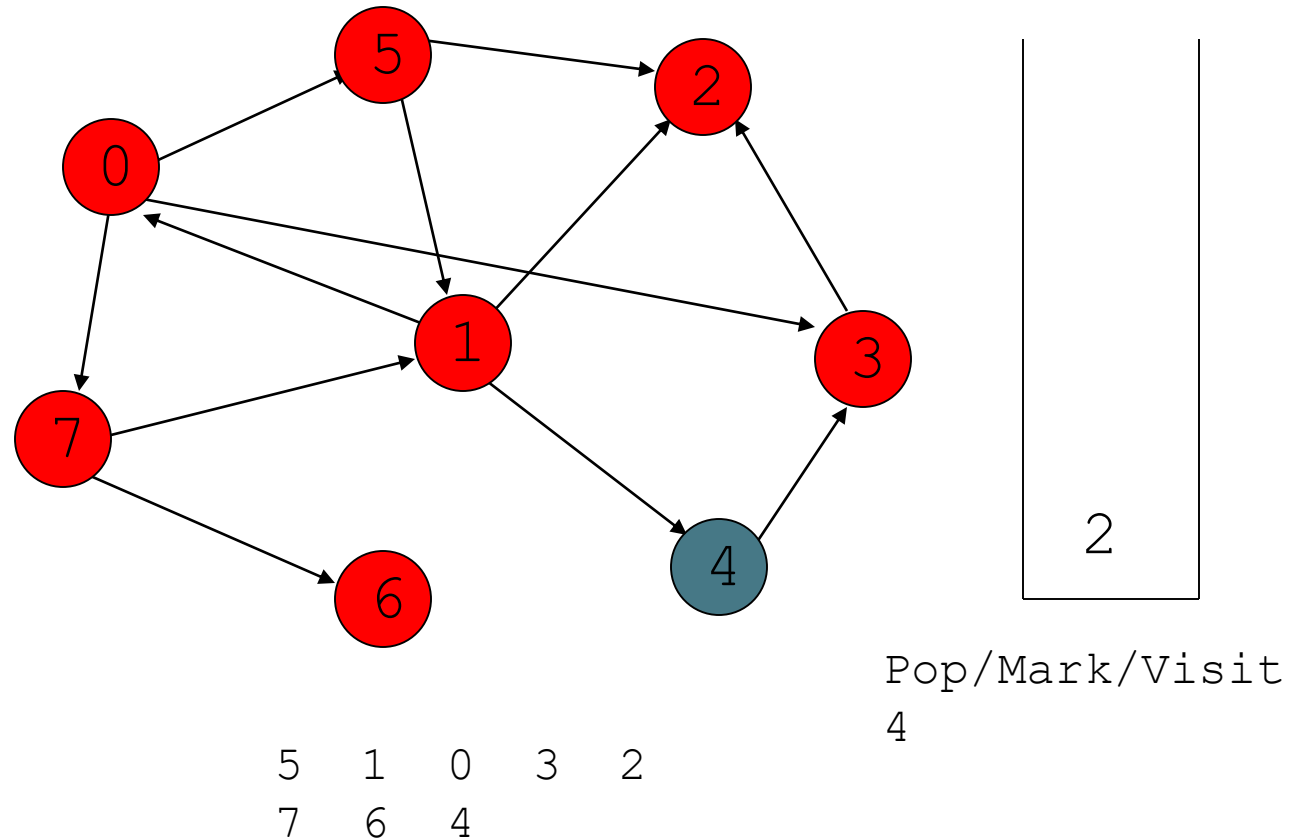
2
4
2

Pop/Mark/Visit
6

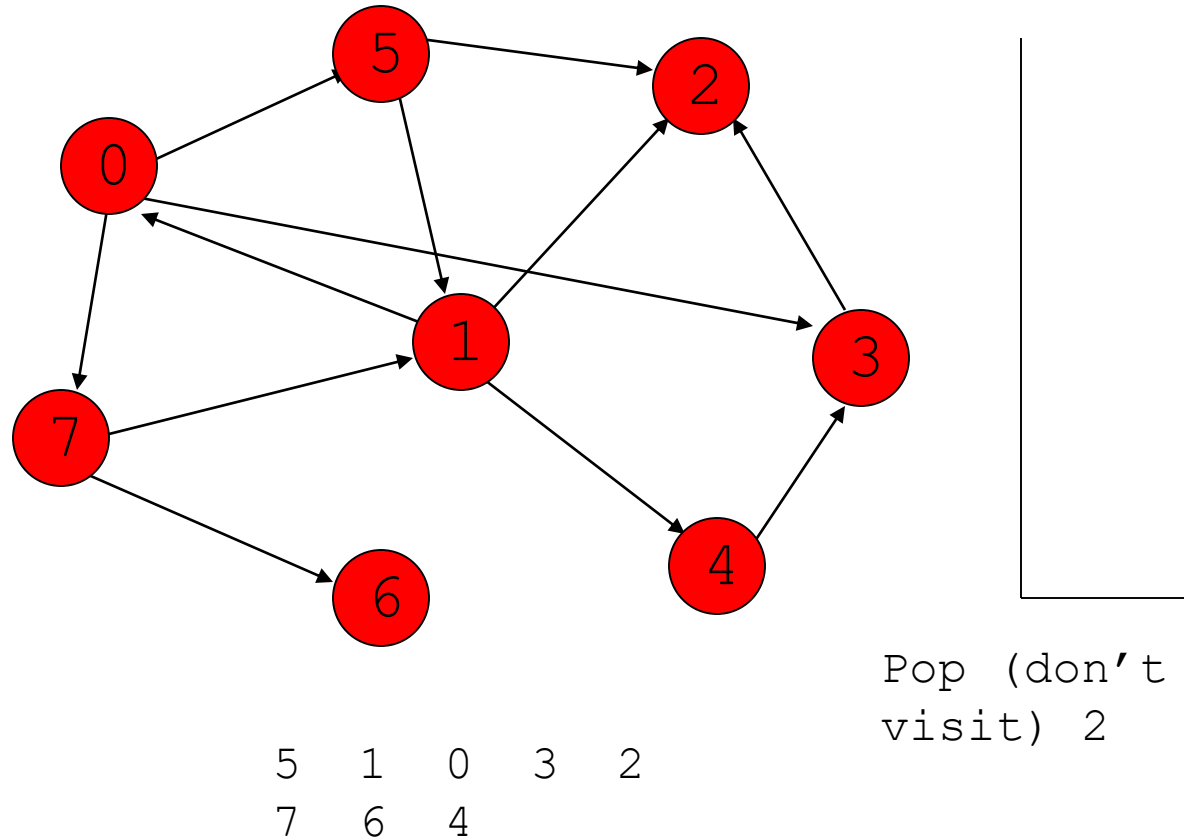
5 1 0 3 2
7 6



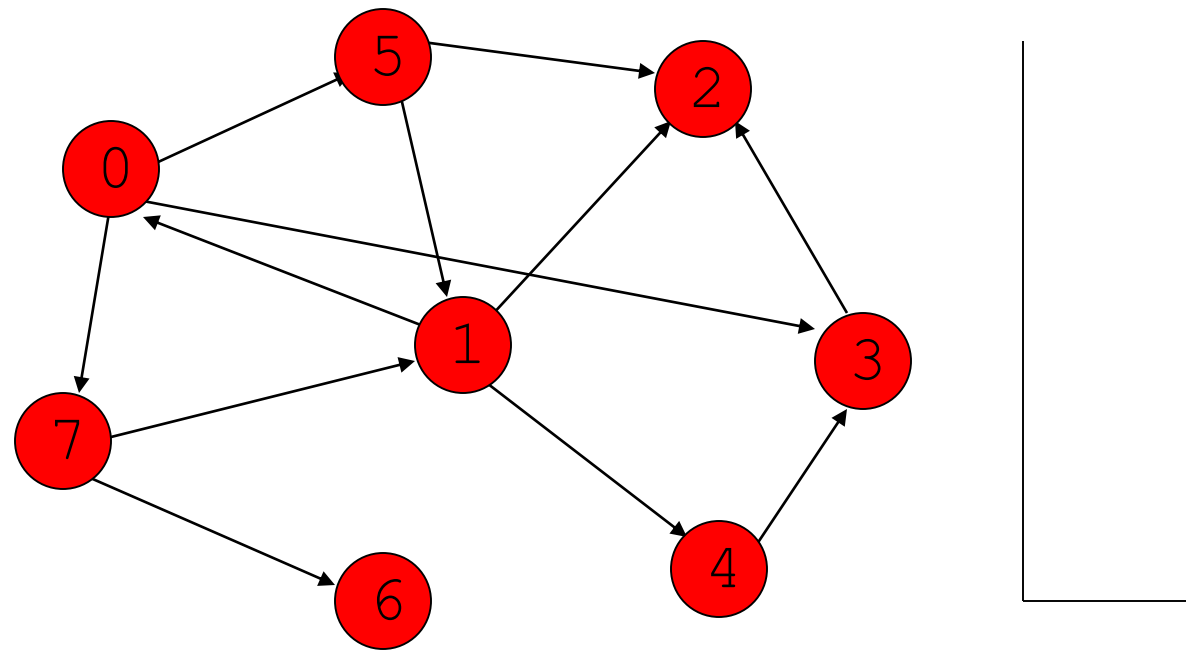
DFS: Start with Node 5



Preorder DFS: Start with Node 5



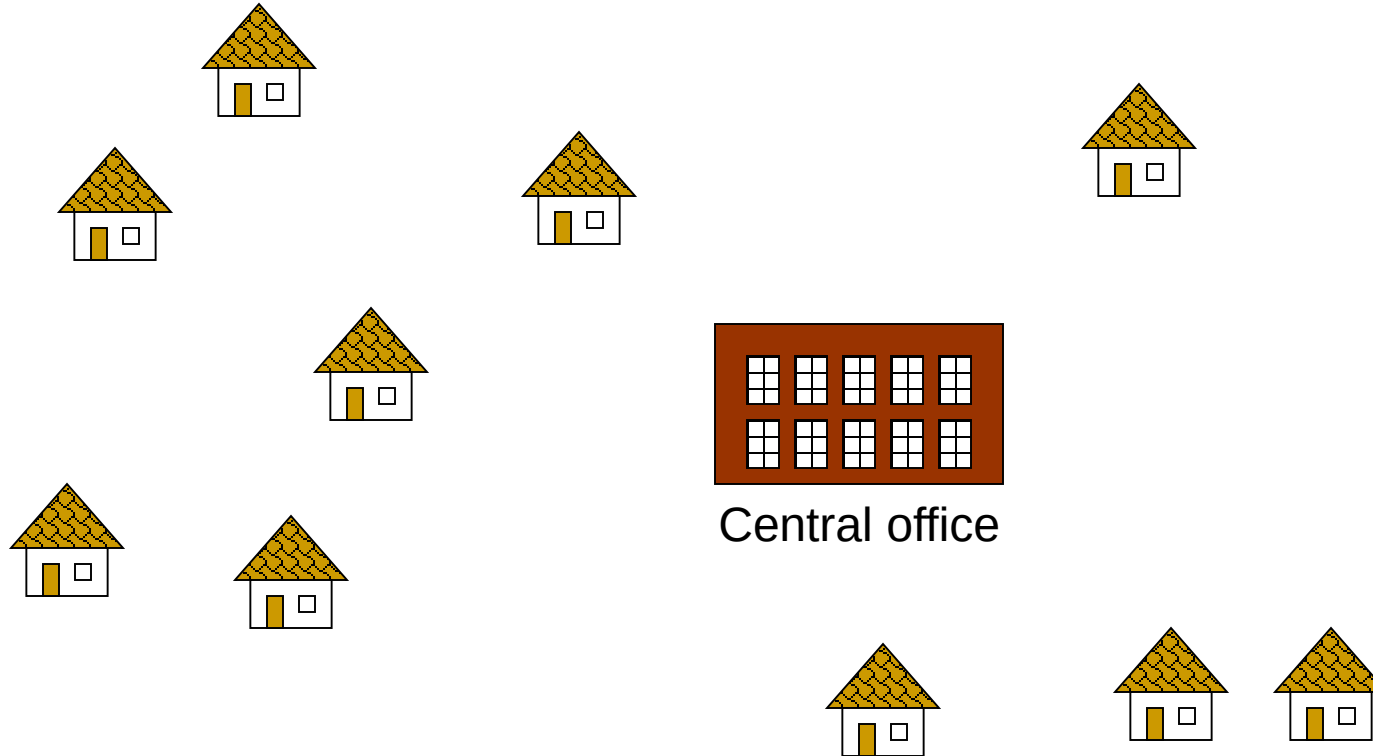
Preorder DFS: Start with Node 5



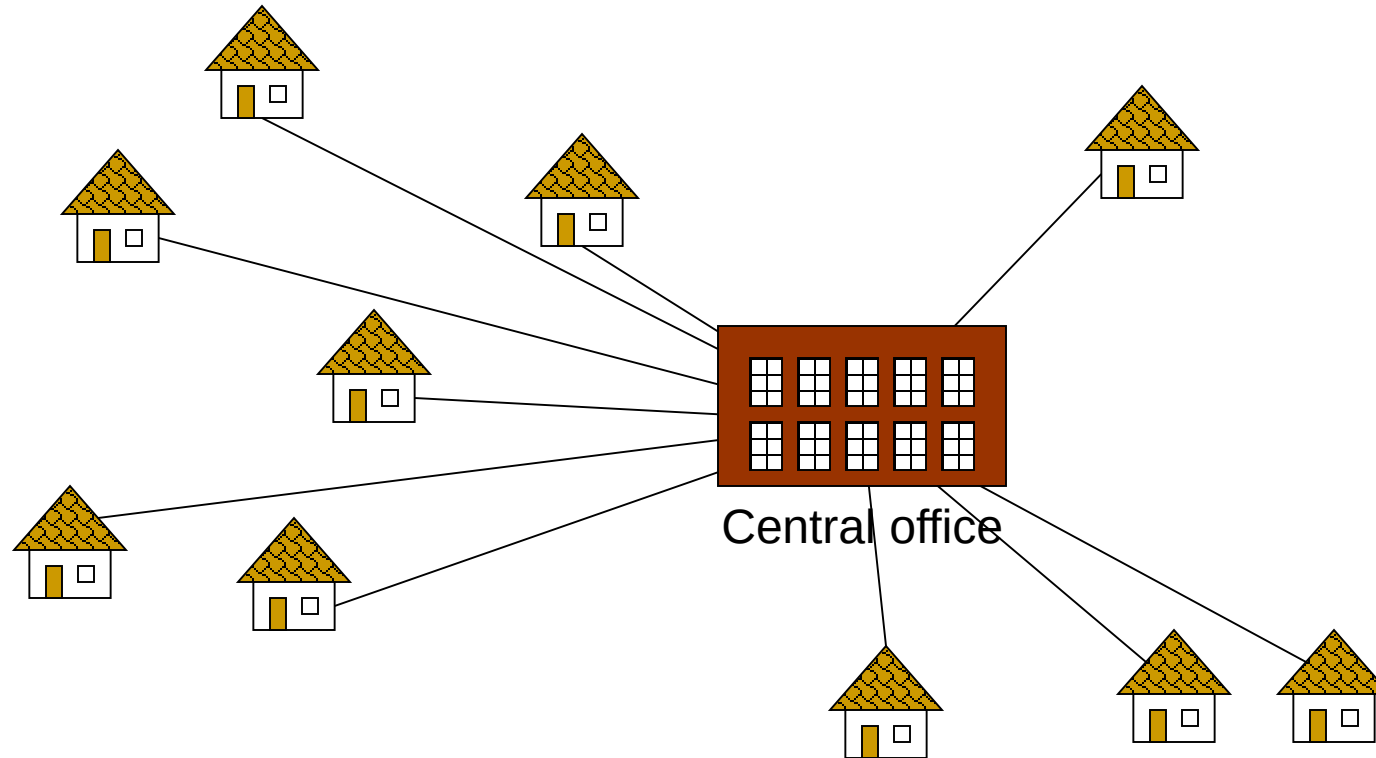
Done

5	1	0	3	2
7	6	4		

Problem: Laying Telephone Wire

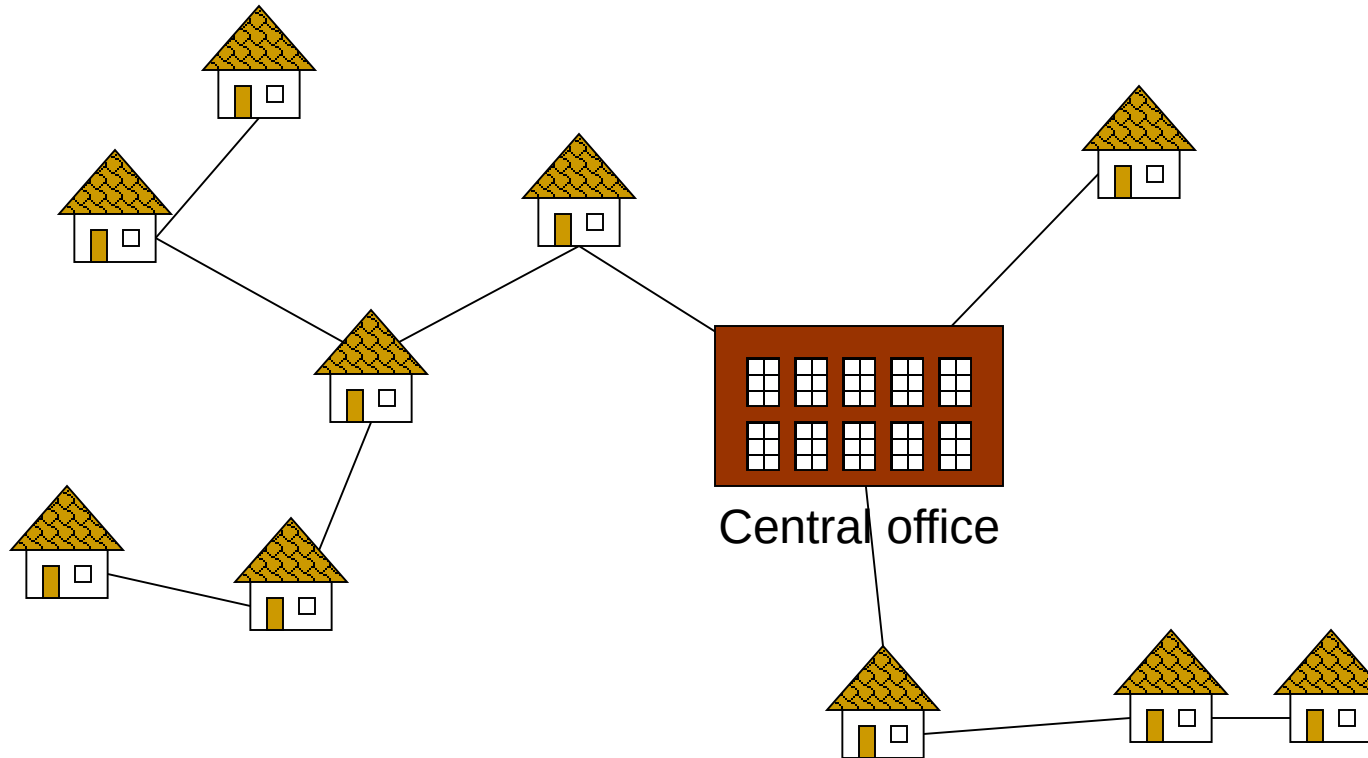


Wiring: Naive Approach



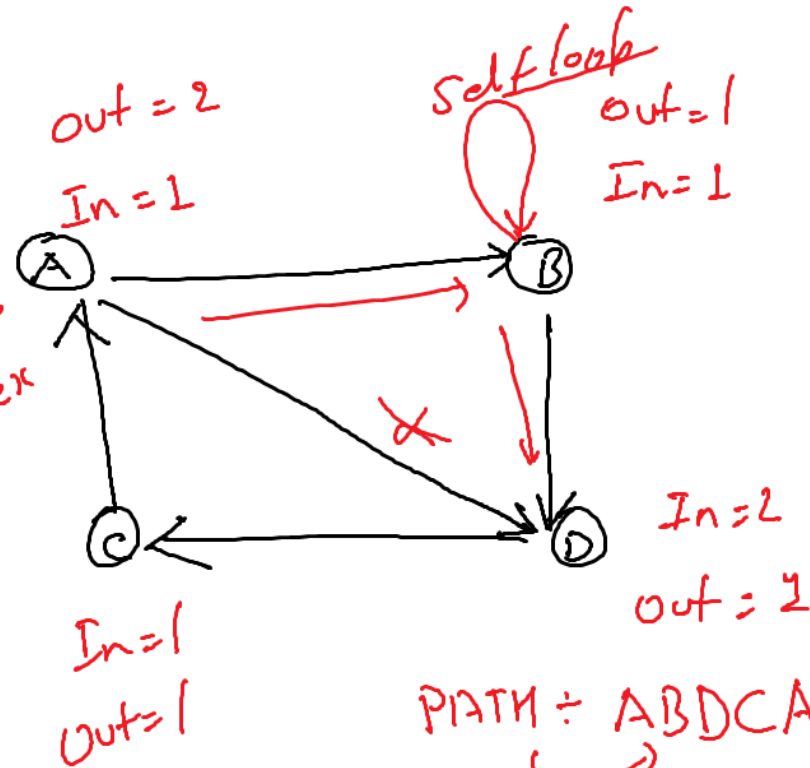
Expensive!

Wiring: Better Approach



Minimize the total length of wire connecting the customers

Reachable
using any no. of
edges you can
reach / traverse
from one vertex
to other.



* Simple path

where we visit
just one loop

PATH: ABDCA
Cycle →

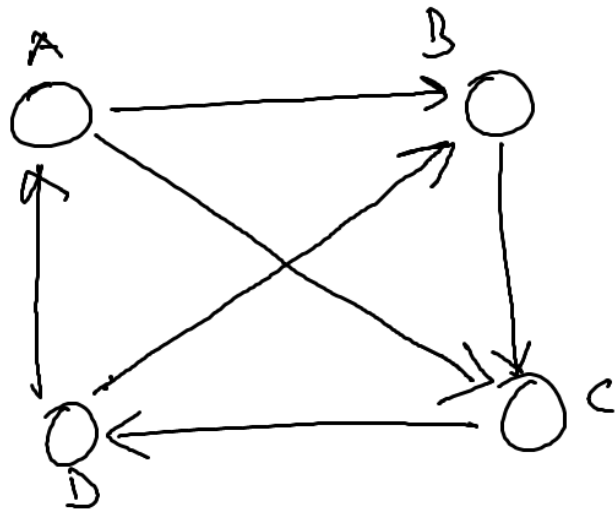
PATH: ABDAX

Source { In = 0
Out = true

No node in this graph
is source

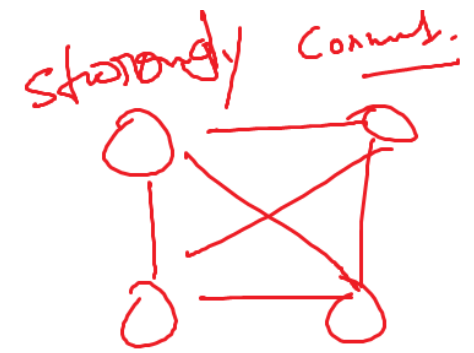
Terminal { In = true
Out = 0

No node in this graph
is terminal.



Connected A B
B C

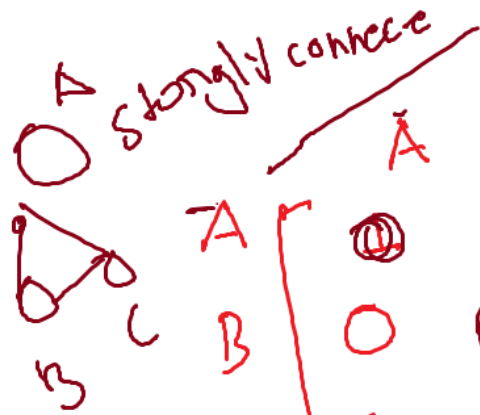
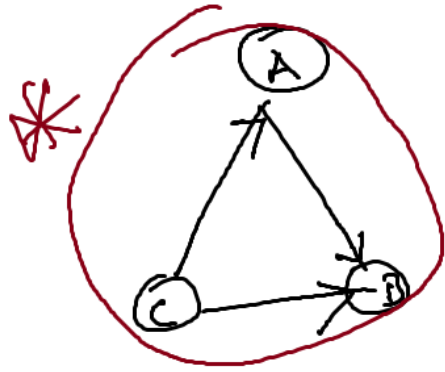
CD
DA



A B C CD DA AB

connected graph: when for any of vertices if we have possibility to reach

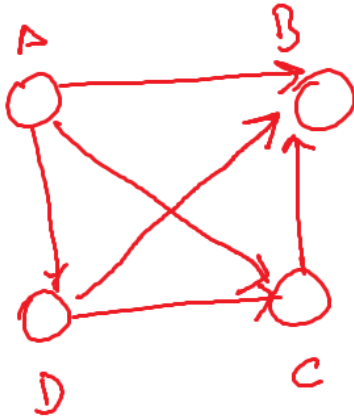
strongly connected: there's direct



	A	B	C
A	1	1	0
B	0	1	0
C	1	1	1

Adjacency matrix

This indicates/identifies the possible of path/edges between adjacent edges



	A	B	C	D
A	1	1	1	1
B	0	1	0	0
C	0	1	1	0
D	0	1	1	1

Binary matrix